PROJECT 4 | CLIMATE

Machine Learning in Climate Analysis and Prediction

Measuring warming, discovering climate zones, and predicting next-day weather events from long-term records.

Team size	2 students	**Duration**	6 weeks
Course weight	_____ % of final grade	**Final due date**	____ / ____ / 20___
Team members	1. _____ 2. _____	**Team number**	_____

1. Project overview

Climate science relies on long, noisy records collected by thousands of stations. In this project your team analyzes temperature records reaching back to 1750, links warming to atmospheric CO_2, groups the world's cities into climate zones without labels, and predicts next-day rain and heat events from daily station observations, including a station near your campus.

You will apply every model studied in the course to trend estimation, forecasting, clustering and event classification.

Research questions your project must answer:

1. How fast has the planet warmed per decade, and has the rate changed since 1975?
2. How accurately can you forecast monthly city temperatures 12 months ahead?
3. Do unsupervised clusters of cities match known climate zones?
4. Can next-day rain and heat days be predicted from today's observations, and are heat days becoming more frequent locally?

2. Learning objectives

By the end of this project you will be able to:

- Clean and aggregate long, gappy time series from multiple sources.
- Separate long-term trend from seasonal cycles with regression and engineered features.
- Use PCA and k-Means to discover structure in data without labels.
- Compare all models studied in the course on the same data using sound validation and appropriate metrics.
- Communicate technical results clearly in a written report and an oral presentation.

3. Datasets

You will use four public datasets.

Dataset	Size	Target	Access
1. Climate Change: Earth Surface Temperature Data (Berkeley Earth)	Monthly land temperatures from 1750 (global), by country, state and city; the city file has millions of rows. Built from 1.6 billion temperature reports	Monthly average temperature (deg C)	Kaggle CSV
2. NOAA Mauna Loa CO2 record	Monthly mean CO_2 from March 1958 to the present (about 820 rows) plus a global series	Used as a predictor (ppm)	Free CSV from NOAA Global Monitoring Laboratory; public domain
3. Rain in Australia	About 145,000 daily observations from 49 locations over about 10 years; 23 variables	RainTomorrow (yes / no)	Kaggle CSV
4. Meteostat station data	Hourly and daily observations (temperature, precipitation, wind, pressure, humidity) for thousands of stations, decades back	Next-day heat day (yes / no); next-day maximum temperature	pip install meteostat; data CC BY 4.0

- Early Berkeley Earth records (before 1850) are sparse and uncertain. Use 1850 onward for trend models and discuss the uncertainty columns.
- Each team chooses 10 cities for forecasting (spread across continents) and one local station for Phase 5, and lists them in the proposal.

4. Required models

Every model below must be applied in your project. Section 5 tells you where each one fits. Wrap preprocessing and model in a scikit-learn *Pipeline* so scaling is learned on training folds only.

Family	Models (all required)
Regression	Linear regression, Ridge, Lasso, Polynomial regression
Classification	Logistic regression, k-Nearest Neighbors, Naive Bayes, Support Vector Machine (SVC / SVR)
Trees and ensembles	Decision tree, Random forest, Gradient boosting
Neural networks	Multilayer perceptron (scikit-learn MLPClassifier / MLPRegressor, or Keras)
Unsupervised	Principal Component Analysis (PCA), k-Means clustering

5. Step-by-step instructions

Phase 1: Setup and exploration (Week 1 to 2)

1. Download all datasets; report coverage (years, locations) and missing values for each.
2. EDA: global annual temperature 1850 to present with a 10-year rolling mean; CO_2 curve; monthly cycle for 3 contrasting cities; rain frequency by Australian location.
3. Compute temperature anomalies relative to a 1951 to 1980 baseline.

Phase 2: Trend analysis (Week 3)

1. Fit linear and polynomial (degree 2 and 3) regression to global annual temperature since 1850. Report warming per decade with a confidence interval.

2. Fit separate linear models for 1850 to 1974 and 1975 to present; compare slopes.
3. From 1958 onward, add annual mean CO_2 as a predictor. Compare Ridge and Lasso models with and without CO_2.

Phase 3: Monthly forecasting (Week 3 to 4)

1. For each of your 10 cities, build features: lags of 1, 2, 3 and 12 months, a 12-month rolling mean, sine and cosine of month, and year.
2. Split by time: train through 2000, test 2001 to 2013. Use TimeSeriesSplit for cross-validation.
3. Train and tune every regression model: linear, Ridge, Lasso, polynomial, k-NN regressor, decision tree, random forest, gradient boosting, SVR, MLP regressor.
4. Compare with a seasonal-naive baseline (same month last year).

Phase 4: Climate zones (Week 4)

1. Describe each city by its 12 average monthly temperatures (1961 to 1990), annual range and latitude.
2. Apply PCA; plot cities on the first two components and interpret them (for example, warmth and seasonality).
3. Run k-Means; choose k with the elbow and silhouette methods. Plot clusters on a world map using latitude and longitude.
4. Compare your clusters with Koppen climate zones for 20 well-known cities.

Phase 5: Next-day event classification (Week 4 to 5)

1. Rain in Australia: handle missing values, encode wind directions and locations, split by date (train to 2015, test 2016 onward).
2. Train and tune all classifiers: logistic regression, k-NN, Naive Bayes, SVM, decision tree, random forest, gradient boosting, MLP. Address the imbalance (about 22% rain days).
3. Local station (Meteostat): download at least 20 years of daily data. Define a heat day as maximum temperature above the 90th percentile of the full record; predict it one day ahead with all classifiers.
4. Count heat days per year and fit a linear trend: are they increasing at your station?

6. Evaluation metrics

Task	Required metrics
Trend and forecasting	RMSE and MAE (deg C), R^2; warming rate per decade with confidence interval; skill vs seasonal-naive baseline
Event classification	Precision, recall, F1, ROC-AUC; confusion matrix; per-location accuracy for Rain in Australia
Clustering	Silhouette score, elbow plot, agreement with Koppen zones

Report all metrics on the **held-out test set**, never on training data. Summarize every model in one comparison table with the columns: model, best hyperparameters, cross-validation score (mean and standard deviation), test metrics, training time and prediction time.

7. Expected deliverables

D1. Jupyter notebook (*ProjectX_Team##.ipynb*)

A single, clean notebook with these sections, in this order:

1. Title, team members and a one-paragraph summary of results
2. Imports, configuration and data loading (with download instructions)

3. Exploratory data analysis (at least 6 plots, each with a 1 to 2 sentence interpretation)
4. Preprocessing and train / validation / test split
5. Unsupervised analysis (PCA and k-Means)
6. Supervised models: one subsection per required model, with cross-validation and hyperparameter tuning
7. Model comparison table and plots (forecast vs actual for 3 cities, ROC curves, climate-zone world map)
8. Interpretation: feature importance, error analysis, answers to the project questions
9. Conclusions and limitations

D2. Written report (PDF, IEEE conference format, 8 to 12 pages)

1. **Abstract** (150 to 200 words)
2. **Introduction:** the problem, why it matters, and your research questions
3. **Related work:** at least 4 cited papers or reports on ML for this problem
4. **Data:** sources, size, features, preprocessing, and the split strategy
5. **Methods:** each model, its key hyperparameters and the tuning search space
6. **Results:** comparison table, key figures, and the best model
7. **Discussion:** answers to the discussion questions in this handout
8. **Conclusion and future work**
9. **References** and **contribution statement**

D3. Presentation

- 10 to 12 slides, 12 minutes plus 3 minutes of questions; both students present.
- Must include: problem, data, method, the model comparison table, best model, one live or recorded demo of a prediction, and lessons learned.

D4. GitHub repository

```
ProjectX_Team##/
  README.md            # project summary, how to get the data, how to run
  requirements.txt     # pinned package versions
  notebooks/           # the final notebook (and optional drafts)
  src/                 # optional helper functions (.py)
  data/README.md       # download instructions only; do NOT commit large data
  figures/             # PNG figures used in the report and slides
  report/              # final report PDF
  slides/              # final slides PDF
```

Final submission checklist

- ☐ Notebook runs top to bottom without errors and has all 9 sections
- ☐ Every required model is trained, cross-validated and tuned
- ☐ PCA and k-Means analyses are included and interpreted
- ☐ Comparison table reports all required metrics on the held-out test set
- ☐ Warming rate per decade is reported with a confidence interval, and the local heat-day trend is analyzed
- ☐ Report is 8 to 12 pages, IEEE format, with references and contribution statement
- ☐ Slides uploaded as PDF; presentation rehearsed to 12 minutes
- ☐ GitHub repository is shared with the instructor and contains no raw data files

8. Timeline and milestones

Week	Milestone	What to submit
1	Kick-off: data access and proposal	1-page proposal: team members, research questions, data loaded (screenshot of df.shape and df.head())
2	Exploratory data analysis and preprocessing	Notebook checkpoint: EDA section and clean, split data
3	Baseline models	Notebook checkpoint: at least 4 models trained with cross-validation
4	All models tuned	Notebook checkpoint: every required model tuned; draft comparison table
5	Interpretation and draft report	Draft report (sections 1 to 5) for instructor feedback
6	Final submission and presentation	Final notebook, report, slides and GitHub link; 12-minute presentation

9. Grading rubric

Criterion	Weight	What an excellent submission shows
Data understanding and EDA	15%	Clear description of size, features, target and class balance; at least 6 meaningful, labelled plots; data quality issues found and explained
Preprocessing and validation	15%	Correct handling of missing values, encoding and scaling inside a Pipeline; correct split (time-based where required); no data leakage
Implementation and tuning of all models	25%	Every required model implemented, cross-validated and tuned with GridSearchCV or RandomizedSearchCV; code is clean, commented and reproducible (fixed random_state)
Evaluation, comparison and interpretation	20%	One comparison table with the required metrics; best model justified; feature importance and error analysis; honest discussion of limitations
Written report	15%	IEEE format, 8 to 12 pages, all required sections, figures numbered and referenced, sources cited
Presentation and individual questions	10%	Clear 12-minute talk shared by both members; each student can explain any part of the code and results

10. Discussion questions (answer in the report)

1. Does a strong statistical relationship between CO_2 and temperature prove causation? What else would you need?
2. Why do forecasts get worse further into the future, and which models degrade fastest?
3. How do changes in instruments, station locations and missing early data bias long-term trends?
4. How could your heat-day model support public health or energy planning in your region?

11. Team rules

- **Team size:** exactly two students. Both students must write code, appear in the commit history and present.
- **Contribution statement:** the report ends with a short table listing who did what. Unequal contribution may lead to different individual grades.
- **Reproducibility:** the notebook must run top to bottom on a clean environment (Google Colab or Kaggle Notebooks) without manual steps, using fixed random seeds.

- **Large data:** you may sample large datasets, but you must state the sample size, how it was drawn (random, stratified, by time) and why.
- **Academic integrity:** cite every dataset, paper, tutorial or code snippet you use. Copying another team's or online project's code without citation is plagiarism. Follow the course policy on AI assistants.
- **Late submissions:** follow the course syllabus policy.

12. Getting started: starter code

Install the packages, then load the data. Adapt paths to your environment.

```
pip install numpy pandas scikit-learn imbalanced-learn matplotlib seaborn \
            kaggle meteostat
```

```
import pandas as pd

# Berkeley Earth (Kaggle: berkeleyearth/climate-change-earth-surface-temperature-data)
glob   = pd.read_csv("GlobalTemperatures.csv", parse_dates=["dt"])
cities = pd.read_csv("GlobalLandTemperaturesByCity.csv", parse_dates=["dt"])
annual = (glob.set_index("dt")["LandAndOceanAverageTemperature"]
              .resample("YS").mean().loc["1850":])

# NOAA Mauna Loa CO2 monthly means
# (download co2_mm_mlo.csv from gml.noaa.gov/ccgg/trends/data.html)
co2 = pd.read_csv("co2_mm_mlo.csv", comment="#")

# Rain in Australia (Kaggle: jsphyg/weather-dataset-rattle-package)
rain = pd.read_csv("weatherAUS.csv", parse_dates=["Date"])
print(rain["RainTomorrow"].value_counts(normalize=True))

# Meteostat (1.x API): daily data for a station near campus
from datetime import datetime
from meteostat import Point, Daily
site = Point(18.47, -66.11)        # replace with your campus coordinates (lat, lon)
daily = Daily(site, datetime(2000, 1, 1), datetime(2025, 12, 31)).fetch()
```

Tuning template used for every model:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV, TimeSeriesSplit
from sklearn.ensemble import RandomForestRegressor

pipe = Pipeline([("scaler", StandardScaler()),
                 ("model", RandomForestRegressor(random_state=42))])
grid = {"model__n_estimators": [200, 500], "model__max_depth": [None, 10, 20]}
cv = TimeSeriesSplit(n_splits=5)          # never shuffle time series
search = GridSearchCV(pipe, grid, cv=cv, scoring="neg_root_mean_squared_error", n_jobs=-1)
search.fit(X_train, y_train)
print(search.best_params_, -search.best_score_)
```

13. Tips for success

- Work with anomalies rather than raw temperatures when comparing cities in different climates.
- The Berkeley Earth city file is large; load only the columns you need (usecols) and filter cities early.
- Check the current Meteostat documentation if the API differs from the starter code.
- Start with a small sample to debug your pipeline quickly, then scale up.
- Commit to GitHub at least twice a week; the history is part of the evidence of teamwork.

Sources

- Kaggle / Berkeley Earth: Climate Change, Earth Surface Temperature Data
- NOAA Global Monitoring Laboratory: Trends in Atmospheric Carbon Dioxide
- Kaggle: Rain in Australia
- Meteostat: open weather and climate data